

REPORTE TÉCNICO



García Torres Daniela Catalina
Espinoza Robles Fátima Gabriela
Recamier Acevedo José de Jesús
Lizaola Hernández Aylin Danae
Lomelí Mejía María del Sagrario
Ferrer Saucedo Valeria Lizeth

ARQUITECTURA DE COMPUTADORAS

I. INTRODUCCIÓN Y ANTECEDENTES

La filosofía de diseño RISC (*Reduced Instruction Set Computer*) marcó un hito en la evolución de la arquitectura de computadoras al demostrar que un conjunto de instrucciones simplificado, de longitud fija y con ejecución en un solo ciclo de reloj por instrucción base, optimiza drásticamente el rendimiento del hardware.

La arquitectura MIPS (*Microprocessor without Interlocked Pipeline Stages*) de 32 bits es el estándar académico e industrial para estudiar este modelo. Este proyecto aborda de forma integral el codiseño de hardware y software, uniendo una herramienta de compilación gráfica con la descripción estructural de la ruta de datos del procesador.

II. OBJETIVO DEL PROYECTO

Diseñar, describir e implementar de forma modular un procesador funcional basado en la arquitectura MIPS de 32 bits utilizando el lenguaje de descripción de hardware Verilog, validando su comportamiento mediante simulación testbench, y desarrollando simultáneamente un software traductor en Python para automatizar la conversión de código ensamblador a lenguaje máquina binario.

III. MARCO TEÓRICO E ISA (CONJUNTO DE INSTRUCCIONES)

El procesador opera bajo un espacio de direcciones estructurado en palabras de 32 bits y gestiona de forma nativa un banco de 32 registros de propósito general. El conjunto de instrucciones (ISA) implementado se divide en tres formatos específicos:

1. **Tipo R (Registro):** Operaciones aritmético-lógicas puras entre registros. Su código de operación (*Opcode*) es siempre `000000` y la operación exacta la dictamina el campo *funct*. Incluye: `add`, `sub`, `and`, `or`, `sllt`.
2. **Tipo I (Inmediato):** Operaciones que involucran un operando constante de 16 bits (con extensión de signo). Incluye instrucciones de transferencia de datos (`lw`, `sw`), operaciones aritméticas intermedias (`addi`) y saltos condicionales (`beq`, `bne`).
3. **Tipo J (Salto):** Transferencias incondicionales de control utilizando una dirección de destino absoluta de 26 bits. Soporta las instrucciones `j` y `jal`.

IV. DESARROLLO DE SOFTWARE: HERRAMIENTA CONVERSORA (**convertidor.py**)

Para enlazar el código de alto nivel con el hardware simulado, se construyó una aplicación con interfaz gráfica de usuario (GUI) en Python utilizando la biblioteca estándar **tkinter**.

- **Procesamiento de Sintaxis:** El programa lee líneas de código ensamblador MIPS (por entrada manual o cargando un archivo plano), elimina espacios en blanco redundantes y comentarios, y separa los operandos.
- **Mapeo de Registros:** Traduce las etiquetas nemónicas de los 32 registros estándar (desde **\$zero** hasta **\$ra**) a sus respectivos vectores binarios de 5 bits mediante un diccionario de equivalencias.
- **Generación de Código Máquina:** Ensambla dinámicamente los campos requeridos por el ISA (Opcode, rs, rt, rd, shamt, funct e inmediatos) entregando cadenas estructuradas de 32 bits.
- **Formatos de Exportación:** Permite guardar los resultados directamente como archivos planos **.txt**, archivos de respaldo **.asm** o archivos de configuración de memoria **.mem** listos para ser leídos por directivas de Verilog como **\$readmemb**.

V. DESARROLLO DE HARDWARE: ARQUITECTURA MODULAR EN VERILOG

El diseño del procesador se estructuró de forma modular en Verilog, donde cada componente físico del *Datapath* monociclo corresponde a un archivo específico interconectado de manera jerárquica en el archivo maestro **Single_Datapath.v**:

1. Unidad de Control Principal (**control_unite.v**)

Recibe los 6 bits del campo *Opcode* de la instrucción y activa las líneas lógicas que configuran los multiplexores y habilitaciones de escritura/lectura del sistema. Soportando señales clave como: **RegDst**, **ALUSrc**, **RegWrite**, **MemRead**, **MemWrite**, **mementoReg**, **Branch**, **BranchNE**, **Jump** y **JumpLink**.

2. Banco de Registros (**b_registros.v**)

Implementa la matriz interna de almacenamiento del procesador (**reg [31:0] registers[0:31]**). Cuenta con dos puertos de lectura asíncronos independientes controlados por los buses **read_reg1** y **read_reg2**. La escritura se realiza de forma síncrona en el flanco de subida del reloj (**posedge clk**) bajo la condición de

que la señal `RegWrite` esté activa y el registro destino sea diferente de `$zero`. Se inicializa por defecto cargando los datos binarios del archivo `TestF1_BReg.mem`.

3. Unidad Aritmético Lógica y Control ALU (`alu.v` y `Alu_Control.v`)

- **ALU Control:** Decodifica las señales globales de `ALUOp` y los bits de `funct` para generar el bus de control de la ALU de 4 bits (`alu_ctrl`).
- **ALU:** Ejecuta la operación física sobre los operandos `a` y `b`. El módulo soporta operaciones AND (`4'b0000`), OR (`4'b0001`), Suma (`4'b0010`), Desplazamiento a la izquierda `sll` (`4'b0011`), Desplazamiento a la derecha `srl` (`4'b0100`), Resta (`4'b0110`) y Comparaciones con signo/sin signo `slt` / `sltu` (`4'b0111` y `4'b1000`). Cuenta con una salida complementaria `zero` que se activa si el resultado de la operación es nulo.

4. Gestión de Memorias (`instruction_memory.v` y `DataMemory.v`)

- **Memoria de Instrucciones:** Actúa como una memoria ROM de 32 palabras donde se aloja el código binario generado por el software conversor; entrega de forma asíncrona la instrucción correspondiente según la dirección del PC.
- **Memoria de Datos:** Representa la memoria RAM dinámica del sistema, con capacidad para 128 posiciones de 32 bits. Realiza lecturas asíncronas cuando se activa `MemRead` y escrituras síncronas en el flanco de subida del reloj si `MemWrite` está habilitado.

5. Componentes de Soporte de la Ruta de Datos

- **PC (`pc.v` y `pc_add.v`):** El contador de programa (`pc.v`) actualiza la dirección en cada ciclo de reloj o se limpia a cero si se activa la línea de `reset`. El submódulo `pc_add.v` suma constantemente 4 unidades de manera asíncrona al PC actual para apuntar a la palabra contigua.
- **Módulos de Extensión y Desplazamiento (`Sign_Extend.v`, `Shift_left.v`, `Adder.v`):** El extensor de signo convierte los inmediatos de 16 bits a valores de 32 bits respetando el bit más significativo. El módulo `Shift_left.v` realiza un corrimiento de 2 bits a la izquierda (multiplicación por 4) para ajustar el offset de los saltos relativos, los cuales se calculan usando el sumador genérico `adder` (`Adder.v`).
- **Multiplexores (`mux2a1.v`):** Módulo parametrizado que conmuta dinámicamente los flujos de datos en puntos críticos de decisión (ej. elegir entre la salida de la ALU o la memoria de datos, o seleccionar el segundo operando de la ALU entre el registro o el inmediato).

(Nota arquitectónica: La estructura interna reflejada en el módulo integrador *Single_Datapath.v* expone además la declaración de registros de segmentación interetapas como *IFID_PC4*, *IFID_instruction*, *IDEX_rs*, *IDEX_rt*, preparando estructuralmente el diseño para una futura evolución hacia una arquitectura con Pipeline).

VI. VALIDACIÓN Y RESULTADOS DE SIMULACIÓN

La verificación del correcto funcionamiento de la integración del hardware se realiza mediante el banco de pruebas avanzado *Single_Datapath_tb.v*:

- **Generación de Estímulos:** Genera una señal de reloj oscilatoria constante con un periodo de 20ns (*always #10 clk = ~clk;*) y aplica un pulso inicial de *reset* durante los primeros 4 ciclos para garantizar que los registros y el PC comiencen en un estado conocido y estable.
- **Monitoreo de Variables:** El testbench mapea internamente cadenas de texto legibles para los nombres de los registros MIPS (asociando de forma visual los índices numéricos a etiquetas como *\$zero*, *\$at*, *\$v0*, *\$t0*, etc.), facilitando la legibilidad del rastreo en la consola de simulación.
- **Verificación de Resultados en Memoria:** Al completarse de forma estricta los 12 ciclos de reloj definidos para la ejecución del programa de prueba, el testbench ejecuta de manera automática una tarea especializada llamada *imprimir_memoria_final*. Esta rutina barre el espacio direccionable de la RAM de datos y despliega en la consola **únicamente aquellas celdas cuyo contenido sea diferente de cero**, mostrando su valor tanto en formato decimal con signo como en formato hexadecimal. Esto proporciona una prueba inequívoca de que las operaciones de carga (*lw*), procesamiento aritmético y almacenamiento en memoria (*sw*) se coordinaron perfectamente a través de las señales de control de la ruta de datos.

VII. CONCLUSIONES

La metodología de diseño modular empleada para el procesador MIPS de 32 bits permitió aislar, codificar y validar individualmente cada una de las subetapas del procesador (Búsqueda, Decodificación, Ejecución, Acceso a Memoria y Escritura).

La estrecha concordancia entre la herramienta lógica en Python y los módulos lógicos en Verilog subraya la importancia crítica de contar con entornos de compilación y simulación robustos en el codiseño moderno de sistemas embebidos y computacionales de alto rendimiento, logrando un procesador monociclo completamente funcional y verificado.

VIII. REFERENCIAS

- Arm Ltd. (s. f.). What is Instruction Set Architecture (ISA)? Arm | The Architecture For The Digital World. <https://www.arm.com/glossary/isa>
- Arm Ltd. (s. f.). What is Instruction Set Architecture (ISA)? Arm | The Architecture For The Digital World. <https://www.arm.com/glossary/isa>
- MIPS architecture overview. (s. f.). <https://tams.informatik.uni-hamburg.de/applets/hades/webdemos/mips.html>
- Spiryn, J. (s. f.). What Is RISC and CISC Architecture & Their Differences. <https://conclusive.tech/glossary/what-is-risc-and-cisc-architecture-their-differences/>